หลักฐานข้อ 1

Test coverage อย่างน้อย 60%

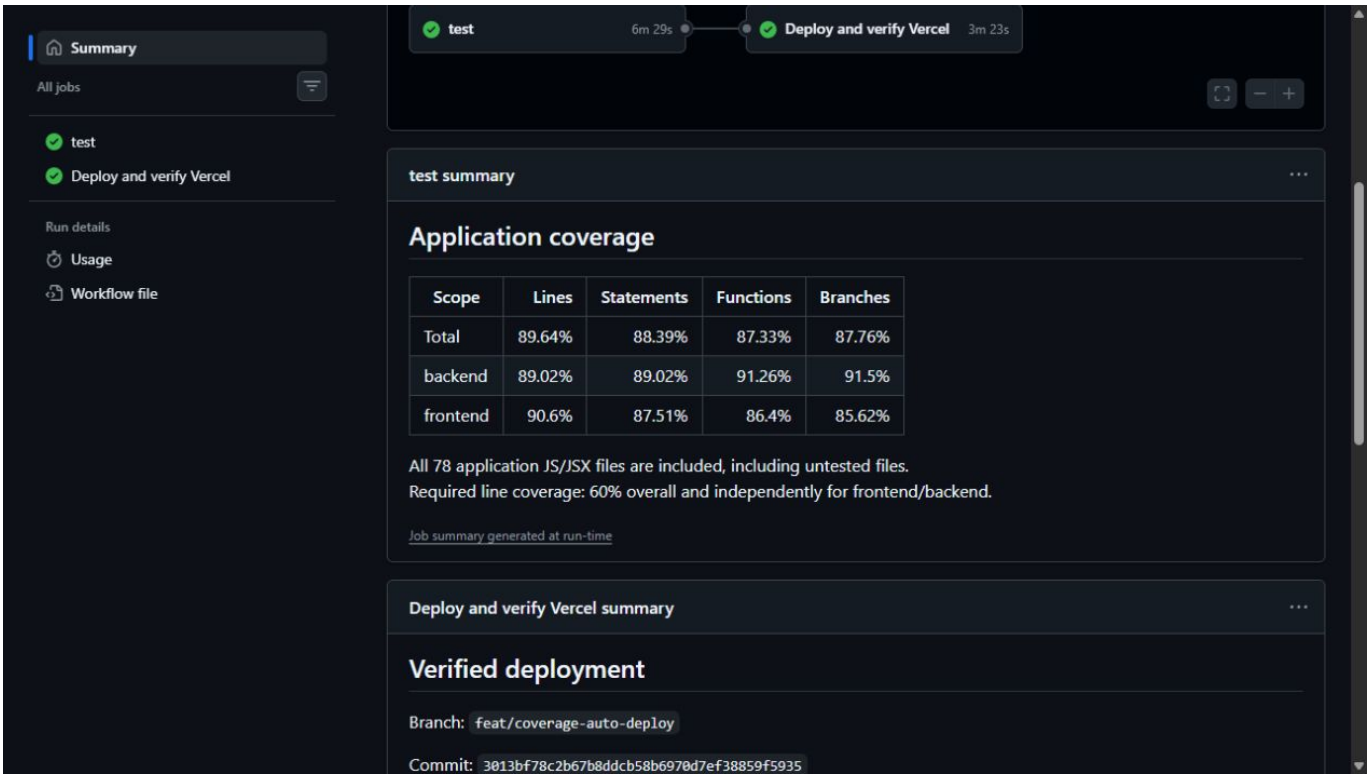
ผ่าน - มีหลักฐานจาก GitHub Actions

89.64%

Line coverage รวมจาก CI บน GitHub - นับทุกไฟล์ JS/JSX ของแอป รวมไฟล์ที่ไม่ได้ถูกทดสอบ

ขอบเขต	CI: commit 3013bf7	ในเครื่อง: ffafe55
รวม	89.64%	90.68%
Backend	89.02%	90.74%
React	90.6%	90.6%
จำนวนไฟล์แอป	78 ไฟล์	79 ไฟล์

ภาพจริงจาก GitHub Actions



ตัวเลข 89.64% เป็นผลบน CI ของ commit 3013bf7 ส่วน 90.68% เป็นผลในเครื่องของโค้ด Node.js commit ffafe55 จึงแสดงแยกกันอย่างชัดเจน

ตรวจผลและตาราง coverage ใน GitHub Actions run #110

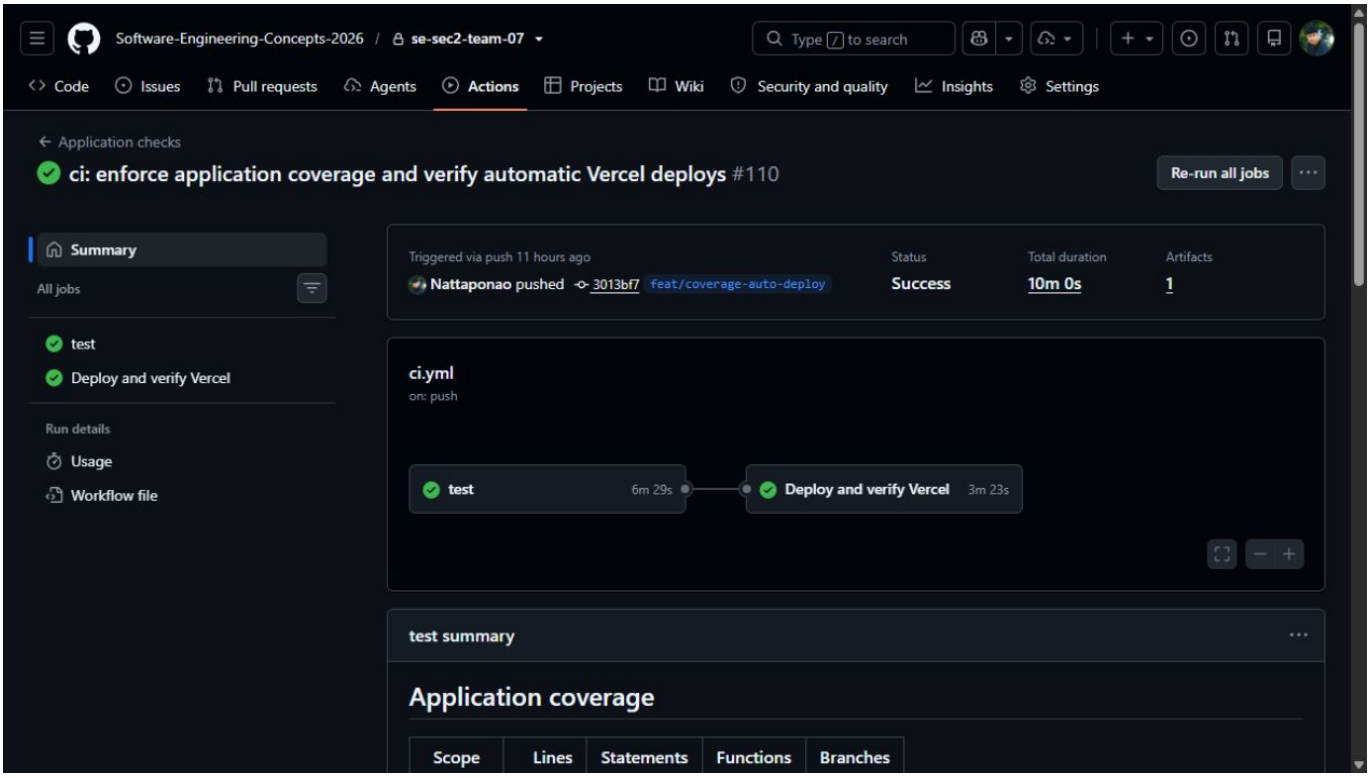
ดาวน์โหลด application-coverage artifact

เกณฑ์ที่ workflow บังคับ: line coverage อย่างน้อย 60% ทั้งภาพรวม Backend และ Frontend ไม่ได้ใช้เฉพาะไฟล์ที่มี tests

CI/CD deploy อัตโนมัติ

ผ่าน - ทดสอบและ deploy จริงจาก push

GitHub Actions run #110 / commit 3013bf7 / branch feat/coverage-auto-deploy



ขั้นตอน	ผลที่ตรวจยืนยัน
Trigger	push (event ที่เกิดจริงใน run นี้)
ทดสอบและ coverage gate	Success - 6 นาที 29 วินาที
Deploy and verify Vercel	Success - 3 นาที 23 วินาที
รวม	Success - 10 นาที

ลำดับที่รันจริง

push → unit/integration/browser tests + coverage → deploy → ตรวจเว็บ/ฐานข้อมูล/API → ตรวจหน้า React บน desktop และ mobile

run นี้ deploy Preview อัตโนมัติจริง ส่วนการ promote Production ถูกข้ามตามเงื่อนไขเพราะไม่ใช่ branch main หลักฐานนี้ไม่ได้อ้างว่า Node.js commit ffafe55 ถูก deploy จาก CI แล้ว

เปิดผล run #110 บน GitHub

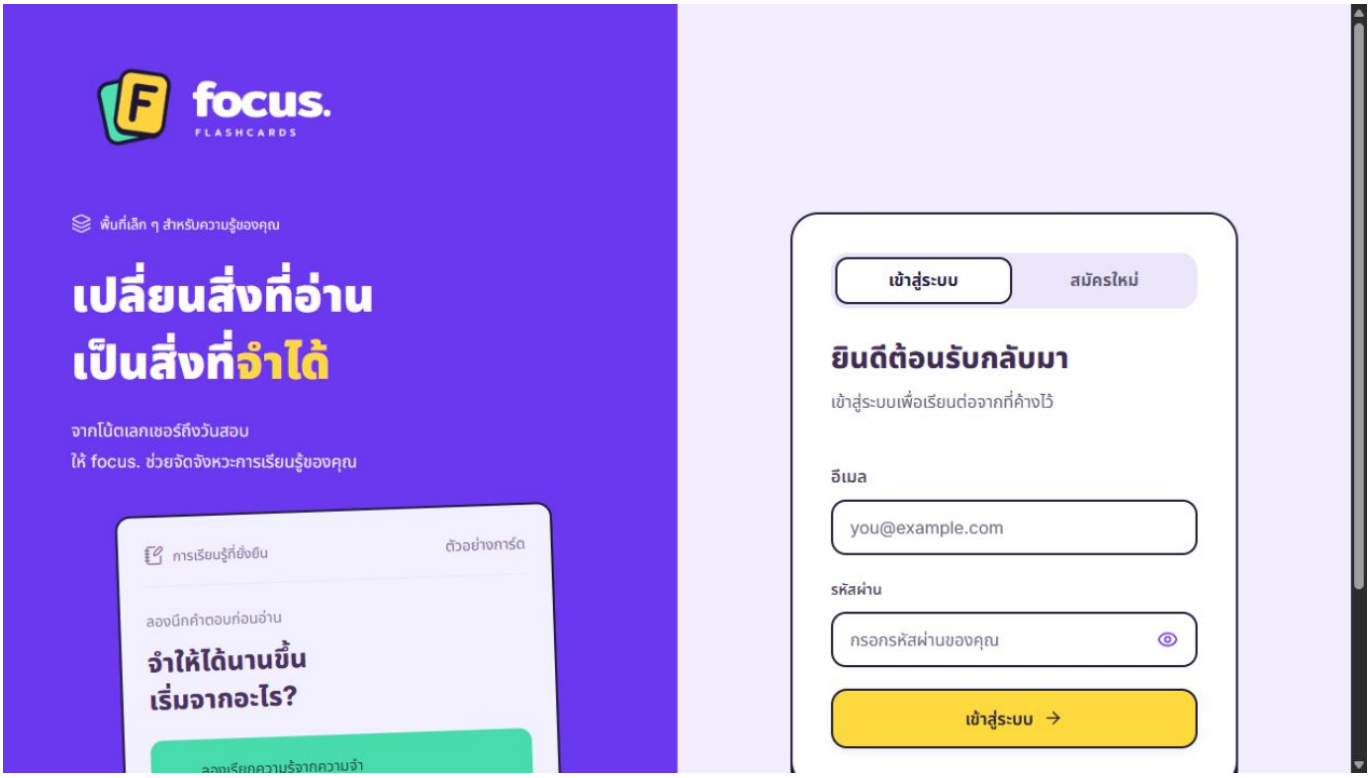
เปิดรายละเอียด job Deploy and verify Vercel

หลักฐานข้อ 2

เว็บที่ deploy ใช้งานได้จริง

ผ่าน - หน้าเว็บแสดงและ live checks ผ่าน

URL ในภาพเป็น Preview ที่ GitHub Actions run #110 สร้างให้อัตโนมัติ ไม่ใช่ localhost



การตรวจ live site	ผล
HTML ของ React และ JS/CSS assets	HTTP 200 / พบแอปและไฟล์จริง
/api/health	HTTP 200 / ฐานข้อมูลตอบ ok=true
/api/me และ /api/decks (ไม่ login)	HTTP 401 ตามเงื่อนไขป้องกันข้อมูล
POST /api/login (JSON ว่าง)	HTTP 400 จาก validation / ไม่ถูกปฏิเสธ origin
Playwright บน CI	desktop/mobile หน้า login และ invalid login ผ่าน

เปิดเว็บไซต์ที่ CI/CD deploy จริง

ตรวจ live checks อีกครั้งวันที่ 11/10/2026 14:12 เวลาไทย และแนบผลจริงใน raw/live-site-check.txt

รุ่น Node.js เป็น Preview ทดลองอีก URL หนึ่ง ทดสอบ login/logout, API ส่วนตัว, TXT/MD/PPTX/PDF/OCR ผ่านแล้วใน session ก่อนหน้า และล้างบัญชีทดสอบเฉพาะครั้งนั้นแล้ว

เปิด Preview ทดลอง Node.js

Response time ต่ำกว่า 2 วินาที

ยังไม่ผ่านครบทุกกรณี - ต้องแสดงผล cold/idle ด้วย

วัดตั้งแต่เริ่มเปิด URL จนเห็นช่องอีเมลของหน้าเข้าสู่ระบบ โดยใช้ Chrome context ใหม่ทุกครั้ง ไม่ใช้เวลาโหลด HTML เพียงอย่างเดียว

เงื่อนไข	Docker Production	Node.js Preview
ค่าสูงสุดในรอบเปิดปกติ	5.438 วินาที	1.225 วินาที
เปิดหลังหยุดค่าขอเกิน 5 นาที	5.547 วินาที	2.532 วินาที
ช่วงพักก่อนรอบดังกล่าว	342 วินาที	352 วินาที
จำนวนรอบที่เกิน 2 วินาที	3 จาก 9 ครั้ง	1 จาก 9 ครั้ง

ผล Node.js Preview ทุกรอบในการวัดชุดเปรียบเทียบ

รอบ	หน้า login พร้อมใช้ (s)	API /me (s)	< 2s
เปิดครั้งแรก	1.225	0.173	ผ่าน
Desktop 1	0.973	0.100	ผ่าน
Mobile 1	0.947	0.111	ผ่าน
Desktop 2	1.017	0.162	ผ่าน
Mobile 2	0.822	0.092	ผ่าน
Desktop 3	0.994	0.117	ผ่าน
Mobile 3	0.929	0.105	ผ่าน
หลังพัก 352 วินาที	2.532	1.413	ไม่ผ่าน
เปิดซ้ำทันที	0.745	0.107	ผ่าน

รอบหลังพักใช้เวลาเปิดหน้า 2.532 วินาที แม้ API /me ตอบใน 1.413 วินาที จึงยังไม่มีหลักฐานรองรับว่าเว็บทุกครั้งต่ำกว่า 2 วินาที งาน OCR หนึ่งหน้าในชุดทดสอบใช้ 3.431 วินาที และไม่ได้ทดสอบเวลาผู้ให้บริการ AI

ข้อจำกัด: Production และ Preview เป็นคนละสภาพแวดล้อม มือถือเป็น viewport จำลอง ไม่มี network/CPU throttling ไม่ใช่ concurrent load test และไม่อาจห้าม traffic จากผู้ใช้อื่นในช่วงพักได้ ผล API ส่วนตัวใช้บัญชีทดสอบว่าง ไม่แทนข้อมูลขนาดใหญ่ของผู้ใช้จริง

วันที่วัด response time: 11/10/2026 เวลา 03:19-03:47 น. (ไทย) ข้อมูลเต็มอยู่ใน raw/production-response.json และ raw/node-preview-response.json พร้อมระยะเวลาทุกรอบ รวมรอบที่เกินเกณฑ์